Web Technology Practical Programs

Full-Stack Web Development & Modern JavaScript Standards

PLACEHOLDER DOCUMENT NOTICE

This is a verified placeholder document. To replace it with your actual PDF file,
save your original PDF as "public/pdfs/web-technology.pdf" in your project directory.

Table of Practical Programs:

1. Responsive Multi-Column Layout using CSS Grid and Flexbox
2. Dynamic Form Validation and Asynchronous DOM Updates
3. RESTful API Client with Fetch API, Async/Await and Error Handling
4. Client-Side Persistent State Management using Web Storage API
5. Interactive Single Page Canvas Graphics & Charting Visualizer

Lab 1: Modern CSS Grid and Flexbox Architecture

Objective: Build a fully responsive fluid card grid without hardcoded media queries.

```
/* Modern Responsive Grid without Breakpoints */
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 1.5rem;
  padding: 2rem;
}

.card {
  display: flex;
  flex-direction: column;
  background: #ffffff;
  border: 1px solid rgba(0, 0, 0, 0.08);
  border-radius: 12px;
  padding: 1.5rem;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05);
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.card:hover {
  transform: translateY(-2px);
  box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.08);
}
```

Lab 2: Dynamic Form Validation and Event Delegation

Objective: Perform instant real-time field validation with custom regex rules.

```
function validateRegistrationForm(formData) {
    const errors = {};
    const emailRegex = /^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$/ ;
    const passwordRegex = /^(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d]{8,}$/ ;

    if (!formData.username || formData.username.trim().length < 3) {
        errors.username = "Username must be at least 3 characters long.";
    }
    if (!emailRegex.test(formData.email)) {
        errors.email = "Please provide a valid email address.";
    }
    if (!passwordRegex.test(formData.password)) {
        errors.password = "Password must be at least 8 chars with 1 number.";
    }

    return {
        isValid: Object.keys(errors).length === 0,
        errors
    };
}
```

Lab 3: Asynchronous Fetch API & JSON Processing

Objective: Fetch paginated REST data with retry logic and error resilience.

```
async function fetchWithRetry(url, options = {}, retries = 3) {
    for (let attempt = 1; attempt <= retries; attempt++) {
        try {
            const response = await fetch(url, {
                ...options,
                headers: { "Content-Type": "application/json", ...options.headers },
            });
            if (!response.ok) {
                throw new Error(`HTTP error status: ${response.status}`);
            }
            return await response.json();
        } catch (err) {
            if (attempt === retries) throw err;
            const backoff = Math.pow(2, attempt) * 100;
            await new Promise(res => setTimeout(res, backoff));
        }
    }
}
```

Lab 4: LocalStorage & Client-Side Session Cache

Objective: Implement offline cache with TTL expiration.

```
class StorageCache {
    static set(key, value, ttlSeconds = 3600) {
        const record = {
            value,
            expiresAt: Date.now() + ttlSeconds * 1000,
        };
        localStorage.setItem(key, JSON.stringify(record));
    }

    static get(key) {
        const raw = localStorage.getItem(key);
        if (!raw) return null;
        const record = JSON.parse(raw);
        if (Date.now() > record.expiresAt) {
            localStorage.removeItem(key);
            return null;
        }
        return record.value;
    }
}
```